

Case Study Challenge | Web Development

Business Context and Case Overview

In the modern urban landscape, while digital connectivity is at an all-time high, physical community engagement is often fragmented across disparate platforms. **Neighbourly** is a hyper-local marketplace designed to bridge this gap, allowing community members to exchange skills, tools, and services (e.g., local tutoring, equipment rental, or specialized repair work).

As a full-stack engineer, your mission is to architect a system that transitions from a simple community board into a robust, high-performance marketplace. This mirrors real-world engineering challenges where systems must evolve from single-tenant MVPs to distributed architectures capable of handling thousands of concurrent interactions with high reliability.

Key Challenges

- **Trust & Verification:** Implementing robust user profiles and verification systems to ensure community safety.
- **Scalability:** Architecting a system that starts with dozens of users and scales to hundreds of thousands across multiple cities.
- **Real-Time Coordination:** Synchronizing service availability and bookings to prevent double-booking or scheduling conflicts.
- **Data Integrity:** Maintaining accurate, immutable logs for service history, transactions, and user ratings.

Candidates should consider system evolution from a single-community model to a large-scale distributed architecture. Key aspects include database design, API efficiency, asynchronous processing, caching strategies, and security measures to support high concurrency and transactional integrity.

Your Role in This Challenge

You have been tasked with developing the backend service and core frontend for Neighbourly. The project will progress through three distinct stages of evolution, requiring you to make critical decisions regarding database schema, API architecture, and system reliability.

Stage 1: The Neighborhood Pilot - Single Community MVP

Initially, the focus is on a single neighborhood to test the core logic of listing and booking services.

- **Core Entities:** User (Provider or Seeker), Service Listing, and Booking Request.
- **Functionality:** A provider can post a service; a seeker can search and book a slot.
- **Tech Constraints:** * Data can be stored locally (JSON files or SQLite).
 - A simple REST API or CLI to perform CRUD operations.
 - Focus on basic validation (e.g., a user cannot book their own service).

Stage 2: The Urban Expansion - Multi-City Network

Neighbourly now supports 500+ neighborhoods. You must introduce a centralized system that handles regional variations and higher traffic.

- **Architecture:** Move to a relational database (e.g., PostgreSQL or MySQL).
- **Location Services:** Implement **Location-Based Queries** to filter services by proximity.
 - **Geospatial Precision:** Instead of simple coordinate math, implement "Radius-based discovery"

(services within 5km/ 10km/ 25km etc.) using a specific spatial indexing library such as **Uber's H3** or **Google's S2**.

- **Features:**
 - Add REST API endpoints with advanced filtering (by category, neighborhood, and price).
 - Implement **Authentication and Authorization** to distinguish between community members and moderators. Implement **Role-Based Access Control (RBAC)** with hierarchical permissions.
 - Introduce Request Throttling to protect the API from abuse.

Stage 3: The National Scale Up - High Scale & Real Time

The system must now support thousands of concurrent operations and nationwide service exchange.

- **Real-Time Communication & Messaging:**
 - Implement an in-app messaging system using WebSockets or gRPC for instantaneous, two-way communication between Seekers and Providers.
 - Integrate a Message Queue (e.g., RabbitMQ or Redis Pub/Sub) to handle message persistence and to trigger push notifications for users who are currently offline.
- **Optimization:** Introduce **Caching (e.g., Redis)** for frequently searched categories and high-traffic neighborhood listings.
 - Use **Read/Write separation** to handle high-volume search queries.
- **Auditability:** Maintain detailed, immutable audit logs of all status changes, financial transactions, and permission overrides to ensure a transparent and secure marketplace.

Areas for Creativity (The Competitive Edge)

We encourage you to go beyond the technical requirements to drive "sustainable growth":

- **The "Reputation Engine":** Design a creative algorithm for user ratings that rewards long-term reliability over one-off reviews.
- **Dynamic UI/UX:** Build a frontend that adapts its view based on the user's role (Provider vs. Seeker) or geographic location.
- **Intelligent Discovery:** How might you use the data to recommend "trending" services in a specific neighborhood?

Submission Guidelines

Your solution should demonstrate how you think critically about system evolution.

- **Code Repository:** Provide a functional repository (GitHub/Zip). Plus a deployed URL, if available (Bonus points for that!)
- **README:** This document must include:
 - **Design Decisions:** Why did you choose specific tools or patterns?
 - **Data Model:** A diagram or description of your schema.
 - **Evolution Rationale:** How does your Stage 1 design facilitate the move to Stage 3?
 - **Assumptions:** Any business or technical assumptions you made.
- **Presentation & Walkthrough:**
 - **5-Minute Pitch:** Explain your thought process and the v1 to v3 evolution.
 - **Live Demo:** A functional walkthrough of the working product.
 - **Q&A:** A 10-minute combined limit for the presentation and walkthrough, followed by judge inquiries.